7.9.13 提高推流帧率

7.9.12 小节的第一版 `camserver` 在一个循环里依次做三件事：抓一帧、转成 JPEG、写给所有观看者。它正确，但慢：三段时间直接相加。本小节记录后来的提速过程，方法可以归结为四条：先测量，把串行改成流水线，去掉大块拷贝，再按瓶颈减少每帧的工作量。

先测量，再优化。 改动之前先给每一帧计时。`stats_frame()` 分别累加三段耗时：*capture* 是 `read()` 拿到一帧所花的时间，*jpeg* 是 `camproc_jpeg_raw()` 的时间，*send* 是把这一帧写进所有观看者 socket 的时间（被慢客户端阻塞也计在里面）。每 3 秒在串口打印一次平均值，`/status` 也给出同样的数字：

```
camserver: N.N fps, B bytes/frame, K viewer(s), capture/jpeg/send C/J/S ms, payload P kbit/s
```

真机上测得的三段约为 70.8、308.2、199.2 ms。串行时一帧要 70.8 + 308.2 + 199.2 ≈ 578 ms，帧率只有约 1.7 fps。三段里最慢的是 JPEG：640×480 的 RAW10 要在 ARM 上做整数去马赛克、白平衡、gamma 和 DCT 编码，用户程序又是以 `-mcpu=cortex-a53+nofp` 编译的，没有浮点和 NEON。

方法一：三级流水线。 三段工作分别由三个执行流承担：采集线程只负责从 `/dev/video0` 取帧，JPEG 线程只负责转换，主线程负责 HTTP 和发送。它们之间用两个深度为 2 的队列相连。这样相邻的三帧可以在不同的 CPU 核上同时处理：第 n 帧在发送时，第 n+1 帧在编码，第 n+2 帧在采集。稳定后的帧周期从三段之和变成三段中的最大值（图 7.37）：

$$T_{\text{串行}} = t_c + t_j + t_s \approx 578\,\text{ms} \quad\Longrightarrow\quad T_{\text{流水线}} = \max(t_c, t_j, t_s) \approx 308\,\text{ms},$$

理论上限从 1.7 fps 提高到约 3.2 fps；实际值还受调度、内存带宽和 Wi-Fi 波动影响。

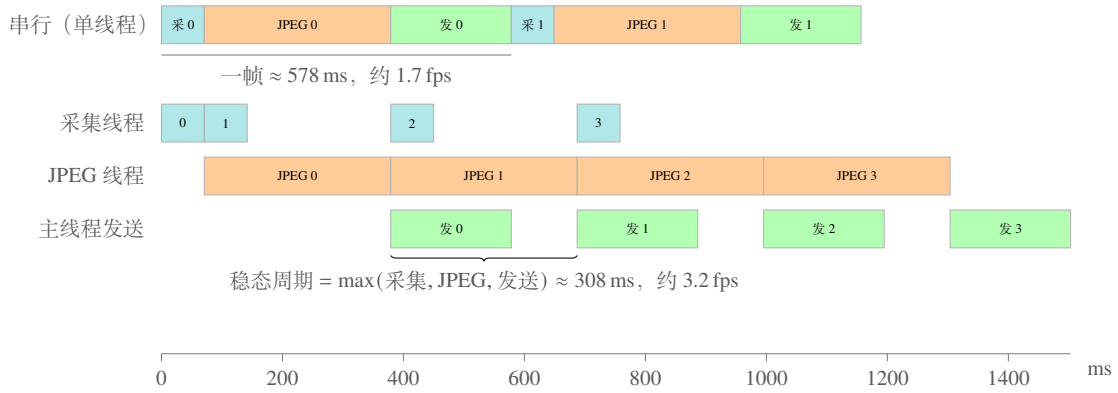


图 7.37: 串行与三级流水线的时间线（按实测的 70.8/308.2/199.2 ms 画，比例尺相同）。流水线中 JPEG 线程几乎一直在工作，帧周期由最慢的这一段决定

为什么用线程而不用进程？xv6 的 TCP 连接记录了接受它的进程号（`c->owner`），已经连上的 socket 不能交给另一个进程。所以接受连接和发送都留在主线程，采集和 JPEG 用 `thread_create()` 创建的用户线程（第 8 章），它们与主线程共享地址空间，可以直接读写同一块缓冲。

方法二：RAW 帧零拷贝。 一帧 RAW10 有 384 KB。第一版用普通 `read()` 把它从内核拷进用户缓冲；现在采集线程只读 40 字节的帧头，JPEG 线程按帧头里的槽号，直接从 `/dev/video0` 的只读 `mmap` 里读像素（7.9.8 小节），每帧省掉一次 384 KB 的拷贝。

零拷贝带来一个所有权问题：内核在同一个文件的下一次 `read()` 时归还上一个槽，如果 JPEG 线程还在读这个槽，像素就可能被 Unicam 的下一帧覆盖。解决办法是打开两个文件描述符，每个各持有一个内核槽，并用信号量 `camera_free[i]` 表示“第 i 个 fd 的槽已经用完”：

```
// 采集线程：轮流使用两个 fd
```